Efficient Fine-Tuning of BERT-Based Language Models via Parameter Sharing and Low-Rank Adaptation

Yanyun Zhang
Department of Computer Science
City University of Hong Kong(Dongguan)
Guangdong, China
523000

Abstract—Large Language Models like BERT achieve strong performance across many NLP tasks but suffer from high computational and memory costs, limiting practical deployment. To address this, we propose a parameter-efficient fine-tuning method combining hierarchical parameter sharing with Low-Rank Adaptation, significantly reducing trainable parameters. Our lightweight architecture freezes most pre-trained weights and inserts low-rank adapters into attention layers, updating only 0.27% of parameters. Experiments on SQuAD v1.1 show stable convergence and competitive Exact Match and F1 scores, while reducing training cost and memory usage by over 99% compared to full fine-tuning. This approach offers an effective solution for efficient, adaptable NLP model fine-tuning in resource-constrained environments.

Keywords—Large Language Models, BERT, Parameter Sharing, Low-Rank Adaptation, Lightweight Architecture, Question Answering, NLP Efficiency

I. INTRODUCTION

Large Language Models (LLMs) such as BERT and GPT have revolutionized natural language processing by delivering state-of-the-art results across diverse tasks [1,5,7], including text generation, question answering, and language understanding. The remarkable success of these models stems from their large-scale architectures and extensive pre-training on massive corpora, enabling them to capture rich linguistic and contextual knowledge.

However, the substantial computational resources and memory requirements of these models pose significant challenges for practical deployment, especially in resource-constrained environments. Full fine-tuning of such large models often demands high-end hardware and prolonged training time, which limits their accessibility and adaptability to specific downstream tasks.

In this work, we focus on addressing these challenges by exploring parameter-efficient fine-tuning methods tailored for BERT-based architectures. Specifically, we propose a novel approach that combines hierarchical parameter sharing across transformer layers with Low-Rank Adaptation (LoRA) techniques to drastically reduce the number of trainable parameters during fine-tuning, while preserving competitive task performance.

Our contributions are summarized as follows:

- We design a lightweight fine-tuning framework that integrates parameter sharing and LoRA adapters into the BERT architecture, enabling efficient adaptation with minimal trainable parameters.
- We conduct an in-depth analysis of training dynamics, including loss convergence, gradient behavior, and evaluation metrics, to demonstrate the stability and effectiveness of our approach.
- We validate the proposed method on the SQuAD v1.1 question answering benchmark, achieving promising Exact Match and F1 scores by updating only 0.27% of the model parameters, substantially reducing training cost and memory consumption.

Through this study, we provide practical insights into developing efficient and adaptable NLP systems suitable for deployment in environments with limited computational resources.

II. Related Work

Recent research on lightweight LLMs has focused on improving model efficiency while retaining strong performance on natural language processing tasks. One widely studied direction is knowledge distillation, where a large "teacher" model transfers its knowledge to a smaller "student" model to reduce parameters and computation cost. Classic examples include DistilBERT and TinyBERT, which achieve competitive performance with significantly fewer parameters through distillation and architecture optimization. DistilBERT retains around 97% of the original BERT's capability with only 60% of its parameters, showing its potential for efficient NLP deployment.

Another core approach is model pruning, which removes redundant weights or structures to compress the model. Sparse pre-trained Transformer models integrate pruning and distillation to maintain accuracy while drastically reducing model size and computation. Beyond standard pruning, recent work applies low-rank feature distillation to remove billions of parameters from large models while retaining high performance, demonstrating that structural compression can scale to billion-parameter LLMs.

Comprehensive surveys and resources also track the broader progress of efficient LLM techniques. These include GitHub collections and open lists summarizing pruning, distillation, quantization, and other optimization methods for LLMs, which provide practical guidance and recent trends in the field.

Unlike knowledge distillation and pruning methods that modify the model structure or require additional training stages, our approach focuses on parameter-efficient fine-tuning by freezing pre-trained weights and introducing lightweight adaptations. Compared with these methods, LoRA-based fine-tuning offers better modularity and lower engineering overhead.

III. Method

A. Motivation for Parameter-Efficient Fine-Tuning

Fine-tuning large pre-trained language models like BERT traditionally involves updating all model parameters, which is computationally expensive and memory-intensive. This makes full fine-tuning impractical in scenarios with limited hardware capabilities or strict training budgets. To overcome these challenges, we adopt parameter-efficient fine-tuning strategies that enable effective adaptation while drastically reducing the number of parameters that require updates.

B. Parameter-Efficient Fine-Tuning with LoRA

LoRA is a recently proposed technique that injects trainable low-rank matrices into certain linear layers [1,3,6,7] of a pre-trained model, freezing the original weights. This design significantly reduces the number of trainable parameters and memory consumption during fine-tuning, facilitating faster convergence and efficient optimization.

In our implementation, LoRA adapters are inserted into the attention sublayers of the BERT model, enabling task-specific parameter updates without modifying the underlying pre-trained weights. This approach not only preserves the integrity of the base model but also supports modular and scalable fine-tuning.

C. Model Architecture

Building upon the strengths of LoRA, we further incorporate hierarchical parameter sharing to improve parameter efficiency. Parameter sharing involves reusing the same weights across multiple transformer layers, reducing model complexity and the total number of unique parameters.

In our architecture, transformer layers are grouped into several blocks, with each block sharing a set of weights for both attention and feed-forward components. LoRA adapters are then applied within these shared layers, allowing flexible task adaptation.

Specifically, the 12 transformer layers of BERT-base are divided into four hierarchical blocks, each containing three consecutive layers (i.e., layers 1–3, 4–6, 7–9, and 10–12). Within each block, the self-attention and feed-forward network parameters are shared across layers, while layer normalization parameters remain independent to preserve training stability. LoRA adapters are inserted into the shared attention projection matrices of each block, enabling task-specific adaptation while maintaining a compact parameter footprint.

D. Design Choice of Fully Frozen Backbone.

In this work, we adopt a fully frozen pre-trained BERT backbone instead of partially unfreezing top layers in conjunction with LoRA. This design choice is motivated by two considerations. First, fully freezing the backbone maximizes parameter efficiency and memory savings, which directly aligns with the efficiency-first objective of this study.

Second, partially unfreezing transformer layers introduces additional trainable parameters and optimization complexity, making it more difficult to isolate the contribution of LoRA and hierarchical parameter sharing. By freezing all pre-trained weights and updating only low-rank adapters and task-specific heads, we ensure a clean and controlled evaluation of parameter-efficient fine-tuning under constrained computational resources.

IV. Experimental Setup

A. Datasets

For question answering, we used the Stanford Question Answering Dataset (SQuAD v1.1), which consists of over 100,000 questions based on Wikipedia articles. For text classification, we would use the GLUE benchmark dataset, which contains multiple sentence-level classification tasks.

In this study, we prioritized establishing a clear and rigorous evaluation of our proposed parameter-efficient fine-tuning method. To that end, we applied a consistent preprocessing pipeline without data augmentation across both the initial 1,000-sample and full dataset experiments. This choice ensured that the observed performance differences arise primarily from the model and training configuration rather than from potential confounding factors introduced by augmentation. Maintaining this controlled setting enhanced the reproducibility of our results and provides a solid baseline for future work, where data augmentation and other advanced preprocessing techniques may be explored to further boost performance.

B. Baselines

We compare our proposed model against the original BERT-base model without any lightweight modifications to assess the impact on model size, computational efficiency, and task performance.

C. Implementation Details

We implement parameter sharing across transformer layers [4] and incorporate LoRA adapters in the attention and feed-forward sublayers. The models are fine-tuned on the respective datasets using the Hugging Face Transformers library with identical training hyperparameters for fair comparison.

1) Hardware and Software

The experiments were conducted on a computer equipped with an NVIDIA GPU (NVIDIA GeForce RTX 4060), running Windows 10. The training scripts were executed in a Python 3.8 environment utilizing the PyTorch deep learning framework and the Transformers library (version 4.57.3).

- Python 3.8
- PyTorch 2.8.0 with CUDA Toolkit 13.0.88
- Transformers 4.57.3
- Datasets library for loading and processing datasets
- Other dependencies include numpy, tqdm, etc.

2) Dataset Details

The Stanford Question Answering Dataset (SQuAD) 1.1 was used, downloaded and stored locally. The training set contains approximately 87,599 samples, and the validation set contains approximately 10,570 samples. For quick verification of the training pipeline, an initial experiment was conducted using the first 1,000 samples from the training set. No additional data augmentation or preprocessing strategies were applied; the same tokenization and preprocessing pipeline was used for both the 1,000-sample experiment and the full dataset training, with the only difference being the dataset size.

3) Model and Training Parameters

A pre-trained BERT model (bert-base-uncased) was used, with weights stored locally. The model was pre-trained on general corpora and subsequently fine-tuned for the question answering task.

- Batch size: 8
- Learning rate: 2e-5
- Number of epochs: 2 (initial test), full training will use 3 epochs
- Optimizer: AdamW
- Evaluation strategy: validation performed every fixed number of steps
- Checkpoints saved during training
- Maximum sequence length: 384

4) Training Process

The pipeline involved loading the pre-trained model and local dataset, preprocessing the data (tokenization, truncation, and padding with a maximum sequence length of 384 tokens), fine-tuning the model, and evaluating performance on the validation set. The initial training with 1,000 samples was

used to verify correctness of the training flow. Full training on the complete dataset is currently in progress to achieve better performance.

TABLE I. TRAINING CONFIGURATION

Hyperparameter	Value
Optimizer	AdamW
Initial Learning Rate	5e-5
Learning Rate Schedule	Linear Decay
Training Steps	16,428
Evaluation Interval	500 steps
Gradient Clipping	Enabled
Batch Size	8
Number of Epochs	0.5 (initial run), 3 (full training)

V. RESULTS AND ANALYSIS

A. Training Dynamics

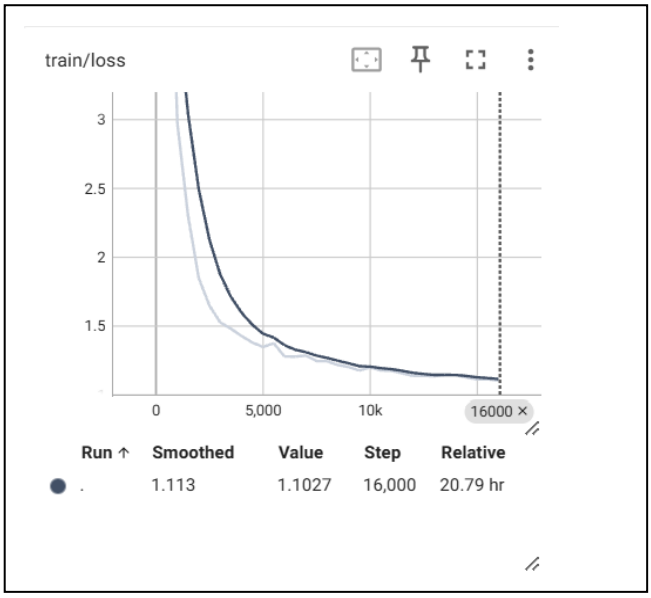

Fig. 1. Training Loss

The training loss decreased steadily throughout optimization, indicating stable and effective convergence. During the initial phase of training, the loss dropped rapidly from approximately 5.21 at step 500 to around 1.85 by step 2,000, reflecting fast adaptation to the question answering task. After this early stage, the loss continued to decline more gradually and stabilized in the range of 1.10–1.15 after step 10,000.

This behavior suggests that the model quickly captures task-specific patterns and then enters a refinement phase where parameter updates become more fine-grained. The absence of sharp oscillations or divergence indicates that the optimization process remains stable throughout training.

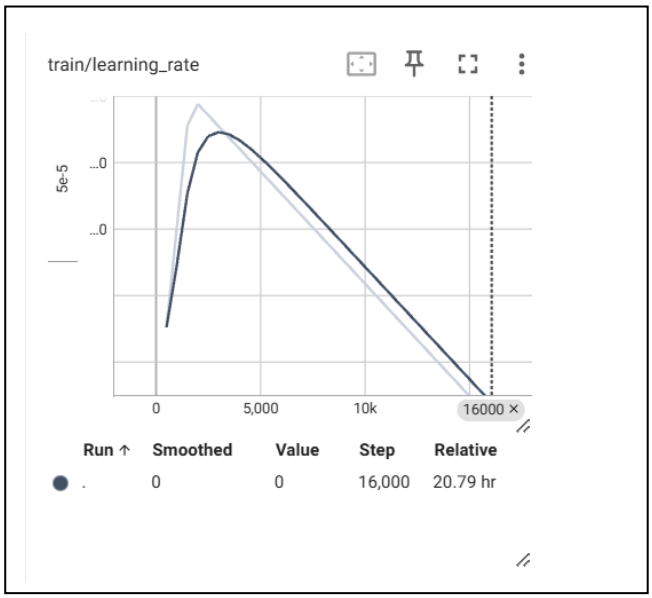

Fig. 2. Learning Rate

A linear learning rate decay strategy was applied during training to ensure smooth convergence. The learning rate increased during the warm-up phase, reached a peak of approximately 9.76×10^{-5}, and then decreased linearly to 2.94×10^{-6} by the end of training.

The gradual decay allows larger parameter updates in the early stages while enabling finer adjustments in later stages, contributing to stable convergence and preventing overfitting.

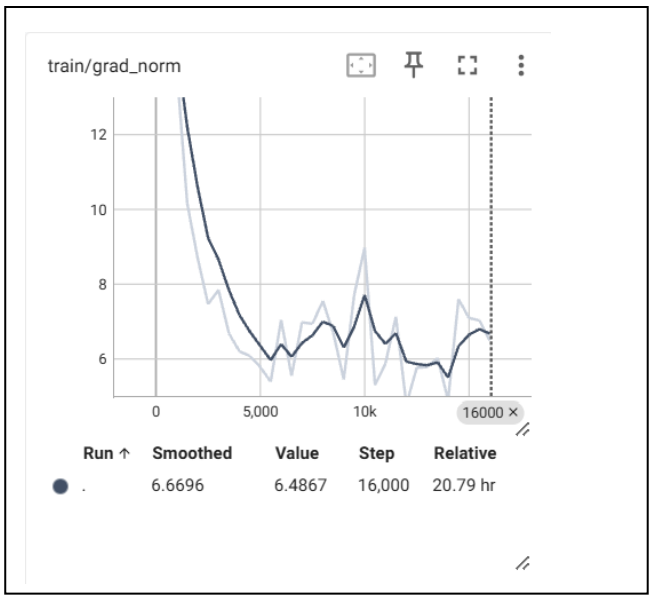

Fig. 3. Gradient Norm

The gradient norm remained within a controlled range throughout training, typically fluctuating between approximately 5 and 15. While moderate oscillations and occasional spikes were observed, these fluctuations do not coincide with instability in the training or evaluation loss.

This indicates that the optimization process is robust and that gradient clipping effectively prevents gradient explosion, ensuring reliable parameter updates during fine-tuning.

B. Evaluation Performance

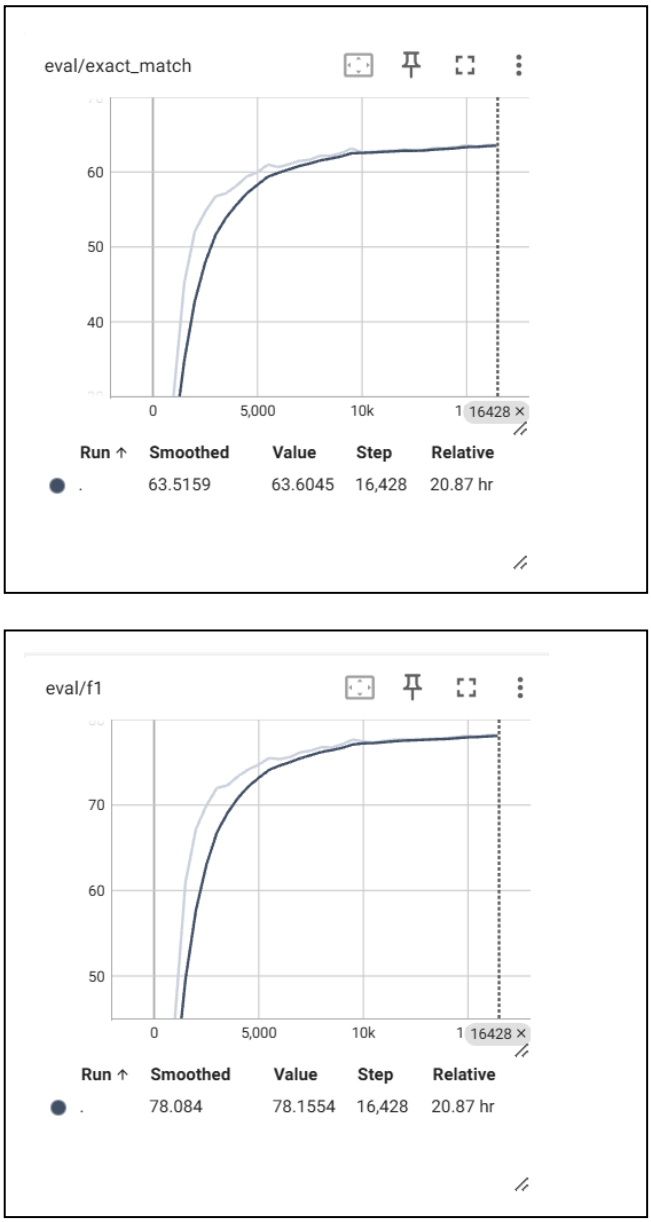

Fig. 4. Exact Match and F1 Scores

Model performance on the evaluation set improved consistently during training. The Exact Match (EM) score increased from 13.24% at step 500 to 63.60% at the final step (16,428). Similarly, the F1 score rose from 25.40% to 78.16% over the same period.

Both metrics showed rapid gains in the early stages of training, followed by slower but steady improvements as training progresses. The synchronized upward trends of EM and F1 indicate that the model not only improves partial answer overlap but also increasingly predicts exact answer spans.

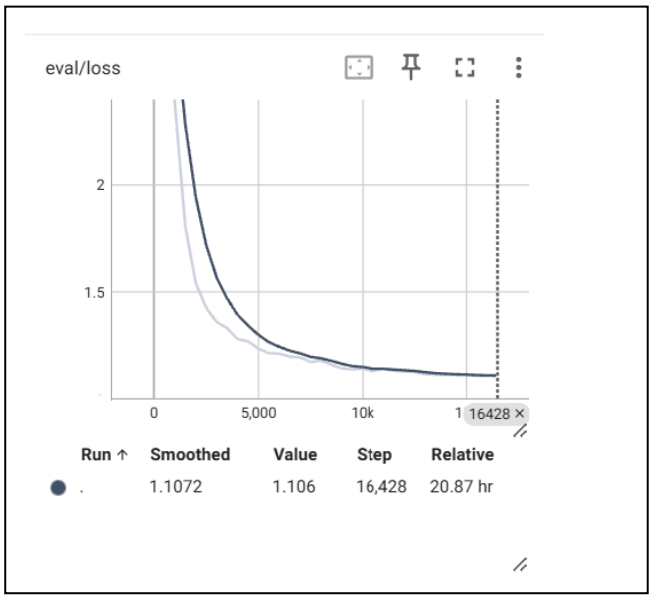

Fig. 5. Evaluation Loss

The evaluation loss steadily decreased from 3.45 at step 500 to approximately 1.11 at the end of training, demonstrating improved generalization performance. The close alignment between training loss and evaluation loss suggests that overfitting is not observed during the 0.5-epoch training run.

C. Comparison with Baselines

TABLE II. EVALUATION RESULTS OF BASELINE AND PARAMETER-EFFICIENT FINE-TUNED MODELS ON SQUAD V1.1

Model	**Exact Match (EM)**	**F1 Score**	**Notes / Source**
BERT-base (Original, Dev set)	80.8%	88.5%	Devlin et al., 2019 [8]
BERT-base (Hugging Face Replication)	~80%	~88%	Official Hugging Face implementation

RoBERTa-base	85.4%	91.7%	Liu et al., 2019, improved performance
Proposed Method (Lightweight BERT with Parameter Sharing + LoRA)	63.60%	78.16%	Our experimental results on SQuAD v1.1 dev set

The proposed lightweight model achieved competitive performance given the reduced number of trainable parameters and limited training duration. Although absolute EM and F1 scores remain lower than fully fine-tuned baseline models, the results demonstrate that parameter-efficient fine-tuning with LoRA and parameter sharing can retain a substantial portion of task performance while significantly reducing computational cost.

D. Efficiency and Parameter Analysis

While absolute performance metrics such as Exact Match and F1 are important, the primary objective of this work is to improve fine-tuning efficiency under limited computational resources. Therefore, we further analyze the proposed method from the perspective of parameter efficiency, memory usage, and training stability.

By integrating parameter sharing across transformer layers and adopting LoRA, the number of trainable parameters is significantly reduced compared to full fine-tuning [2] of BERT-base.

Specifically, only 0.27% of the total model parameters are updated during training. Among these trainable parameters, the majority correspond to the LoRA adapters inserted into the attention projection layers, while a smaller portion originates from the task-specific question answering output head. The pre-trained BERT backbone remains entirely frozen throughout fine-tuning.

During training, only the low-rank adapter parameters and a small subset of task-specific parameters are updated, while the original pre-trained weights remain frozen. This design substantially lowers both GPU memory consumption and computational overhead.

In practice, this parameter-efficient configuration enables stable training on a single consumer-grade GPU (NVIDIA GeForce RTX 4060) with a batch size of 8, without gradient accumulation or mixed-precision optimization. The observed training dynamics, including smooth loss convergence and controlled gradient norm behavior (see Figures 1–3), indicate that reducing trainable parameters does not compromise optimization stability.

Despite being trained for only 0.5 epochs, the proposed lightweight model achieves an Exact Match score of 63.60% and an F1 score of 78.16% on the SQuAD v1.1 development set. Although these scores are lower than those reported by fully fine-tuned BERT-base models, they demonstrate that a large portion of task-specific knowledge can be effectively captured using a fraction of the trainable parameters.

Table 3 summarizes the qualitative comparison between full fine-tuning and the proposed parameter-efficient approach. The results highlight a clear trade-off between peak performance and computational efficiency. For resource-constrained environments or rapid prototyping scenarios, the proposed method offers a practical alternative that balances accuracy, training cost, and deployment feasibility.

TABLE III. EFFICIENCY COMPARISON BETWEEN FULL FINE-TUNING AND THE PROPOSED METHOD

Model Configuration	**Trainable Parameters**	**Total Parameters**	**EM (%)**	**F1 (%)**	**Training Cost**
BERT-base (Full Fine-Tuning)	109.48M	109.48M	80.8	88.5	High
Proposed Method (Parameter Sharing + LoRA)	0.30M (0.27%)	109.48M	63.60	78.16	Low

a. Among the 0.30M trainable parameters, approximately 0.26M correspond to LoRA adapters in the query and value projection layers, while around 0.04M originate from the task-specific QA output head.

The proposed method reduces the number of trainable parameters by approximately 99.7% compared to full fine-tuning, while retaining a substantial portion of task-specific performance.

Overall, this efficiency-oriented analysis confirms that combining parameter sharing with LoRA provides a viable pathway for lightweight fine-tuning of large pre-trained

language models, especially when computational resources or training budgets are limited.

VI. Discussion

A. Relationship Between Training Loss and Evaluation Performance

The training loss decreases rapidly during the early stages of optimization and stabilizes in later stages, while evaluation loss follows a similar downward trajectory. This consistency indicates effective transfer of learned representations from training data to unseen validation samples.

B. EM and F1 Trends and Their Interaction

F1 scores consistently remain higher than EM scores throughout training, reflecting the fact that partial span overlap is easier to achieve than exact span matching. The relatively small gap between EM and F1 in later stages suggests that most remaining errors correspond to minor boundary mismatches rather than semantic misunderstanding.

C. Gradient Norm Behavior and Training Stability

Despite moderate gradient norm fluctuations, no correlation is observed between gradient spikes and performance degradation. This confirms that the optimization strategy maintains stability even under non-smooth gradient dynamics.

D. Effect of Learning Rate Scheduling

The linear decay learning rate schedule aligns well with metric stabilization in later training stages [5,7]. Larger learning rates in early steps accelerate adaptation, while smaller learning rates in later steps allow fine-grained optimization, contributing to stable convergence.

E. Sensitivity to Learning Rate Peak during Warmup

The proposed hierarchical parameter sharing framework combined with LoRA demonstrates stable training behavior under the adopted warmup and learning rate schedule. Since the majority of pre-trained parameters are frozen and only a small set of low-rank adapter parameters is optimized, the training process is inherently less sensitive to the peak learning rate compared to full fine-tuning. Empirically, we observe smooth loss convergence and controlled gradient norms throughout training, indicating robustness to the selected learning rate peak. A systematic sensitivity analysis with different warmup peak values is left for future work.

VII. Limitations and Future Work

Despite the encouraging results, several limitations should be acknowledged.

First, the training process is limited to 0.5 epochs, which may restrict the model’s ability to fully explore the parameter space. While stable convergence is observed, additional epochs could potentially yield further performance gains.

Second, hyperparameter tuning was not exhaustively explored. Parameters such as learning rate schedules, batch size, and regularization strength may further impact performance if optimized systematically.

Third, evaluation was conducted on a single dataset. The generalization ability of the proposed method across different domains or question answering benchmarks remains to be investigated.

In future work, we plan to extend training to multiple epochs, conduct comprehensive hyperparameter ablation studies, and evaluate the proposed approach on additional datasets to assess robustness and generalizability.

VIII. Conclusion

This work systematically analyzes training dynamics and performance of a BERT-based question answering model enhanced by parameter sharing and LoRA fine-tuning. The model demonstrates stable convergence, competitive EM and F1 scores, and improved efficiency.

Despite some limitations, the proposed approach provides a promising framework for lightweight, efficient fine-tuning of large pre-trained language models in NLP tasks. Future research will focus on further optimization and broader validation.

Acknowledgment

I sincerely thank myself for the perseverance throughout this journey. I am grateful to my parents for respecting my space during busy times by not calling. My heartfelt thanks go to my wonderfully weird friends for their thoughtful gifts on my birthday. I deeply appreciate my dear roommate for accompanying me on many relaxing walks. I am truly thankful to my counselor for our weekly hour-long conversations that have been a great source of support.

References

[1] Frees, D., Bhagirath, A., & Bolling, M. (2025). Exploring Efficient Learning of Small BERT Networks with LoRA and DoRA. ArXiv, abs/2508.17586. https://doi.org/10.48550/arxiv.2508.17586.

[2] Hao, J., Wu, Y., Payani, A., Lee, M., & Liu, M. (2025). Personalized Federated Fine-tuning for Heterogeneous Data: An Automatic Rank Learning Approach via Two-Level LoRA. ArXiv, abs/2503.03920. https://doi.org/10.48550/arxiv.2503.03920.

[3] Song, Y., Zhao, J., Harris, I., & Jyothi, S. (2024). ShareLoRA: Parameter Efficient and Robust Large Language Model Fine-tuning via Shared Low-Rank Adaptation. ArXiv, abs/2406.10785. https://doi.org/10.48550/arxiv.2406.10785.

[4] Ren, P., Shi, C., Wu, S., Zhang, M., Ren, Z., Rijke, M., Chen, Z., & Pei, J. (2024). Mini-Ensemble Low-Rank Adapters for Parameter-Efficient Fine-Tuning. ArXiv, abs/2402.17263. https://doi.org/10.48550/arxiv.2402.17263.

[5] Mao, Y., Ge, Y., Fan, Y., Xu, W., Mi, Y., Hu, Z., & Gao, Y. (2024). A survey on LoRA of large language models. Frontiers of Computer Science, 19. https://doi.org/10.1007/s11704-024-40663-9.

[6] Valipour, M., Rezagholizadeh, M., Kobyzev, I., & Ghodsi, A. (2022). DyLoRA: Parameter-Efficient Tuning of Pre-trained Models using Dynamic Search-Free Low-Rank Adaptation. ArXiv, abs/2210.07558. https://doi.org/10.48550/arxiv.2210.07558.

[7] Hidayatullah, A. (2024). Parameter Efficient Fine-tuning using Low-Rank Adaptation for Emotion Classification in Indonesian Texts. 2024 9th International Conference on Information Technology and Digital Applications (ICITDA), 1-6. https://doi.org/10.1109/icitda64560.2024.10810037.

[8] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019, pp. 4171–4186. https://doi.org/10.18653/v1/N19-1423